

Guide de Révision Intensif : Examen Java EE

Basé sur les annales d'examens 4IIR (2022 - 2025)

Ce que vous devez absolument maîtriser :

- Les définitions théoriques (JPA, JDBC, Servlet, MVC)
- Le Cycle de vie des composants
- L'Architecture en 3 Tiers (Présentation, Service, DAO)
- Le Mapping JPA / Hibernate (Les relations et annotations)
- L'Écosystème Spring Boot (Starters, @Query)

Ne lisez pas ce document : Apprenez-le par cur !

Table des matières

1	Théorie : Les Questions de Cours Fréquentes	2
1.1	Concepts Fondamentaux Java EE	2
1.2	Architecture et Composants	3
2	Persistence : JDBC, JPA et Hibernate	4
2.1	L'exercice incontournable : Le Mapping des Relations	4
3	L'Architecture en 3 Tiers (Le Schéma)	6
4	L'Écosystème Spring Boot	7
4.1	Le Formulaire dans Spring MVC (@ModelAttribute)	7

Chapitre 1

Théorie : Les Questions de Cours Fréquentes

Cette section regroupe exactement les questions posées dans la partie théorique de vos examens (notées sur 1 ou 2 points chacune).

1.1 Concepts Fondamentaux Java EE

Q1 : Définir le standard Java EE ?

Le standard **Java EE (désormais Jakarta EE)** est un ensemble de spécifications et d'API fournissant une plateforme robuste, standardisée et distribuée pour le développement et le déploiement d'applications d'entreprise multi-niveaux.

Q2 : C'est quoi le rôle du conteneur WEB (exemple Tomcat) ?

Le conteneur Web est l'environnement d'exécution des composants web (Servlets et JSP). Son rôle est d'intercepter les requêtes HTTP, de gérer le cycle de vie des Servlets (instanciation, exécution, destruction), de gérer les sessions utilisateurs et de renvoyer la réponse HTTP au client.

Q3 : Citer deux exemples de serveurs d'application Java EE ?

- **JBoss EAP / WildFly** (Red Hat)
- **Payara Server** (ou GlassFish)
- **WebSphere** (IBM)

Q4 : C'est quoi la différence entre les méthodes POST et PUT dans HTTP ?

- **POST** : Utilisée pour **créer** une nouvelle ressource. Elle n'est pas idempotente (si on l'exécute 10 fois, elle crée 10 ressources).
- **PUT** : Utilisée pour **mettre à jour** ou remplacer intégralement une ressource existante. Elle est idempotente (l'exécuter 10 fois donne le même résultat final qu'une seule fois).

1.2 Architecture et Composants

Q5 : Expliquer le cycle de vie de la Servlet ?

Il est composé de 3 étapes gérées par le conteneur Web :

1. **init()** : Appelée **une seule fois** lors du chargement de la Servlet en mémoire pour initialiser les ressources.
2. **service()** : Appelée à **chaque requête** cliente. Elle dispatch la requête vers les méthodes spécifiques (doGet, doPost).
3. **destroy()** : Appelée **une seule fois** avant la destruction de la Servlet (arrêt du serveur) pour libérer la mémoire.

Q6 : C'est quoi le problème qui est résolu par le MVC ?

Le MVC (Modèle-Vue-Contrôleur) résout le problème du "Code Spaghetti" en **séparant les responsabilités**. Avant le MVC, la logique métier, l'accès aux données et le code d'affichage (HTML) étaient mélangés dans un même fichier. Le MVC isole l'interface utilisateur (Vue), la logique métier (Modèle) et le routage des requêtes (Contrôleur), rendant l'application maintenable et évolutive.

Q8 : Quel est le rôle de la couche service ?

La couche Service contient toute la **logique métier** de l'application (règles de calcul, vérifications, gestion des transactions). Elle sert d'intermédiaire strict entre le Contrôleur (qui gère le Web) et la couche DAO (qui gère la Base de données).

Chapitre 2

Persistence : JDBC, JPA et Hibernate

Q7 : Quels sont les points forts de JPA par rapport à JDBC ?

- **Productivité (Pas de SQL)** : JPA est un ORM (Object-Relational Mapping). On manipule des objets Java au lieu d'écrire des requêtes SQL complexes et répétitives à la main.
- **Indépendance de la Base de données** : Le code JPA fonctionne sur MySQL, Oracle ou PostgreSQL sans changer une ligne de code (grâce au dialecte configuré).
- **Cache et Performances** : JPA gère automatiquement un cache de premier niveau pour optimiser les accès à la base.

Q13 : Expliquer la différence entre JPA et Hibernate ?

JPA (Java Persistence API) est une **spécification** (une interface). C'est un ensemble de règles et d'annotations (comme `@Entity`) définies par Java EE. Elle ne contient aucun code exécutable.

Hibernate est une **implémentation** de cette spécification (une classe). C'est le framework concret qui exécute le travail et génère le SQL selon les règles dictées par JPA.

2.1 L'exercice incontournable : Le Mapping des Relations

Dans l'examen, on vous donne souvent deux classes (ex : `Article` et `Categorie`) et on vous demande de compléter les lignes (*) pour faire le mapping JPA (OneToMany / ManyToOne).

Règle d'or : La table qui contient la clé étrangère physique dans la base de données est le "propriétaire" de la relation. On y utilise `@JoinColumn`. De l'autre côté, on utilise l'attribut `mappedBy`.

Solution au Mapping (Article et Categorie)

Voici exactement ce que vous devez écrire pour remplacer les (*) :

```
1 // --- Dans la classe Article (Le côté "Many") ---
2 // Un article appartient à UNE seule catégorie. C'est ici que se
   trouve la clé étrangère.
3 @ManyToOne
4 @JoinColumn(name = "categorie_id") // Definit la colonne de
   jointure en base
5 private Categorie categorie;
6
7
```

```
8 // --- Dans la classe Categorie (Le côté "One") ---
9 // Une catégorie contient PLUSIEURS articles.
10 @OneToMany(mappedBy = "categorie") // "categorie" correspond au
    nom de l'attribut dans la classe Article !
11 private List<Article> articles;
```

Chapitre 3

L'Architecture en 3 Tiers (Le Schéma)

Q14 : Compléter le schéma d'architecture 3 tiers

L'examen vous demande d'identifier les classes/interfaces à développer pour chaque couche en utilisant : Servlet, JSP, JPA, et MySQL. Voici la réponse exacte attendue.

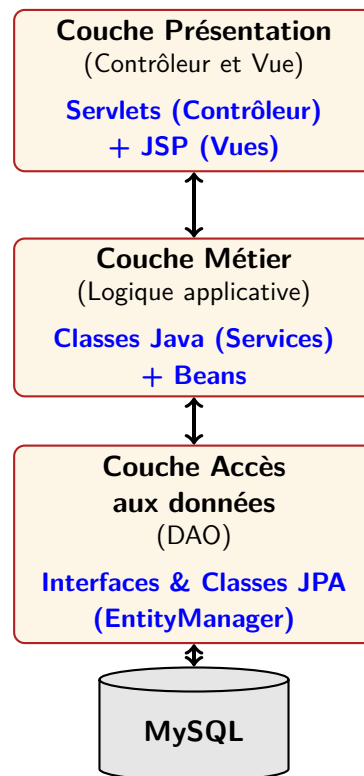


FIGURE 3.1 – La réponse attendue pour le schéma d'architecture

Chapitre 4

L'Écosystème Spring Boot

Q11 : Citer deux exemples de Starter de Spring Boot ?

Les "Starters" sont des dépendances pré-configurées qui facilitent l'initialisation d'un projet. Exemples à citer :

- spring-boot-starter-web (Pour faire du MVC et des API REST)
- spring-boot-starter-data-jpa (Pour utiliser Hibernate et JPA)
- spring-boot-starter-security (Pour la sécurisation)

Q12 : Expliquer le rôle de l'annotation @Query de Spring Data ?

Dans Spring Data JPA, les requêtes sont souvent générées automatiquement à partir du nom des méthodes (ex : findByNom). Cependant, pour des requêtes complexes, l'annotation @Query permet au développeur d'écrire **manuellement** sa propre requête en langage **JPQL** (ou en SQL natif) directement au-dessus de la méthode dans l'interface du Repository.

Exemple concret de l'utilisation de @Query

```
1 @Repository
2 public interface ArticleRepository extends JpaRepository<Article,
3     Integer> {
4     // Exemple d'utilisation de @Query pour une recherche complexe
5     @Query("SELECT a FROM Article a WHERE a.prix > :prix Minimum
6         AND a.categorie.nom = :catNom")
7     List<Article> chercherArticlesChersParCategorie(
8         @Param("prixMinimum") double prix,
9         @Param("catNom") String categorie);
10 }
```

4.1 Le Formulaire dans Spring MVC (@ModelAttribute)

Si l'on vous demande comment récupérer les données d'un formulaire HTML dans Spring Boot :

Définition à retenir

L'annotation `@ModelAttribute` est utilisée dans le Contrôleur Spring. Elle permet de **lier automatiquement (Data Binding)** les données envoyées par un formulaire HTML directement aux attributs d'un Objet Java (ex : `Article`). Cela remplace les fastidieux `request.getParameter(...)`.

```
1 @PostMapping("/sauvegarderArticle")
2 public String saveArticle(@ModelAttribute("article") Article article) {
3     // L'objet "article" est déjà rempli avec les saisies de
4     // l'utilisateur !
5     articleService.save(article);
6     return "redirect:/listeArticles";
7 }
```